

Prototype 1 Build Guide — Overview & Strategy

MAGGIE · Wayne State CSC4996 · Prototype 1 (Weeks 1–4) Audience: the MAGGIE dev team. This guide turns the existing Demo-1 mock app into a real Epic → mapper → Supabase pipeline **without rewriting the screens**, and tells you exactly which files to add, in what order, and how to commit and push them.

1. Where we are vs. where Prototype 1 ends

Demo 1 already shipped a complete, themed **frontend mock flow**: Home → Patient Snapshot → Medication Match → Symptom Check → Session Summary, driven entirely by `src/domain/maggieMockData.ts`. Nothing touches Epic, Supabase, or OpenRouter yet.

Prototype 1 adds the **data pipeline** behind that same UI.

Definition of Done (from `docs/prototype-phases.md`):

A teammate can clone the repo, set env vars, run the app in Expo Go, log into the Epic sandbox, see a FHIR-derived medication/condition list, and confirm a `sessions` row was created.

P1 deliverable	Status today	Slice
Expo / RN + TS skeleton	Done (Demo 1)	—
Epic SMART on FHIR sandbox auth (PKCE)	Not started	Slice A
Retrieve sandbox Patient, Condition, medication resources	Not started	Slice B
Deterministic FHIR → <code>GameContext</code> mapper	Not started	Slice B
Demo screen shows real connected patient	Mock only	Slice B
Supabase project + <code>sessions</code> table + RLS	Not started	Slice C
<code>.env.example</code> + setup docs	Done	—
<code>main</code> / <code>develop</code> / feature workflow active	Partial	Doc 04

2. Guiding principles (do not violate)

These come straight from `docs/architecture.md`, `docs/decisions.md`, and `docs/security-and-scope.md`. Every PR in Prototype 1 is reviewed against them.

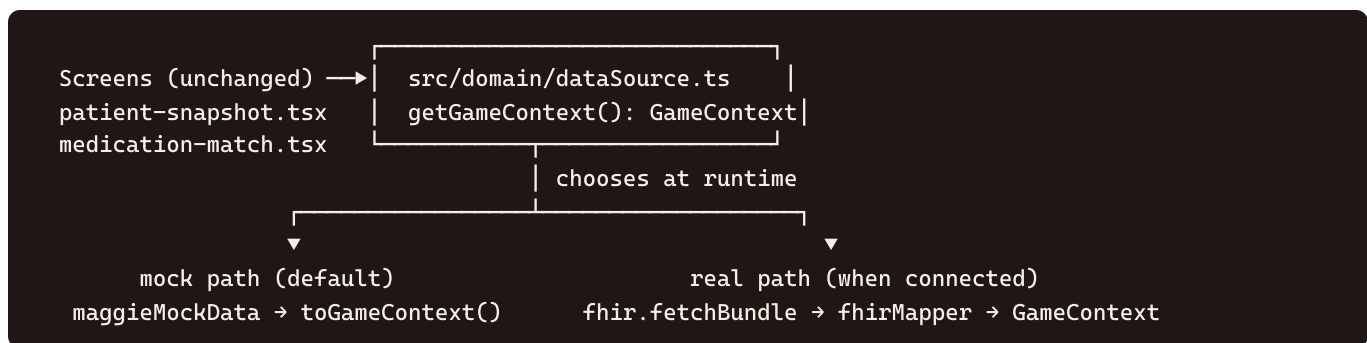
- Keep the existing structure.** We are *adding* `src/services/*` and `src/domain/*` modules, not restructuring `app/`, components, theme, or navigation. Screens keep importing from one data module; only that module changes.
- The mock shapes are the contract.** `maggieMockData.ts` was deliberately written "close to the future `GameContext`." The mapper's job is to produce the *same shapes* from real FHIR so screens never learn

where the data came from.

3. **Deterministic mapper, never the LLM (ADR-005).** FHIR → `GameContext` is plain TypeScript. No LLM in Prototype 1 at all.
4. **Secrets stay server-side.** No `EXPO_PUBLIC_*` secret keys. Epic tokens live only in `expo-secure-store`. Supabase mobile uses the `anon` key with RLS (ADR-003, ADR-004).
5. **Sandbox only.** `EXPO_PUBLIC_EPIC_FHIR_BASE_URL` must point at an Epic sandbox host; never a production FHIR base. No real PHI ever enters git, logs, or screenshots.

3. The "seam": how mock and real coexist

The single most important design decision in Prototype 1 is **introducing one data seam** so the mock and real paths are interchangeable. We do this with a `GameContext` type plus a `dataSource` provider.



- `GameContext` (new, `src/domain/gameContext.ts`) is the shared shape — a superset of what the mock already exposes (profile, conditions, devices, medications, beliefs, symptom sets).
- `dataSource.ts` returns mock data until an Epic session exists, then returns mapped real data. Screens import `getGameContext()` and nothing else changes.
- Because the mock already mirrors the target shape, Slice B is mostly "write the mapper to the type the mock implies," not "redesign the UI."

This seam is what lets the three slices land **in parallel** and be demoed incrementally: Slice C (Supabase) and Slice A (auth) can merge before the mapper is perfect, because the default path is still the mock.

4. Files Prototype 1 adds

Nothing below replaces existing files; it extends the structure documented in `docs/repository-structure.md`.

New file	Slice	Purpose
src/domain/gameContext.ts	B	Shared <code>GameContext</code> type + <code>fromMock()</code> adapter
src/domain/dataSource.ts	B	Runtime provider: mock vs. mapped real
src/services/epicAuth.ts	A	SMART discovery, PKCE, token exchange/refresh
src/services/secureTokens.ts	A	Token read/write via <code>expo-secure-store</code>
src/features/auth/AuthContext.tsx	A	Connection state, exposes <code>connect()</code> / tokens
app/connect-epic.tsx	A	"Connect to Epic (sandbox)" screen
src/services/fhir.ts	B	Typed FHIR reads (Patient, Condition, meds)
src/domain/fhirMapper.ts	B	Deterministic FHIR bundle → <code>GameContext</code>
src/domain/__fixtures__/*.json	B	Saved sandbox bundles for mapper tests
src/services/supabase.ts	C	Supabase client from anon key
src/features/session/sessionService.ts	C	Create/track a <code>sessions</code> row
supabase/migrations/0001_sessions.sql	C	<code>sessions</code> table + RLS

5. Dependencies to add

All Expo-managed, SDK 55 compatible. Install with `bunx expo install` (it pins SDK-correct versions — do **not** `bun add` these directly, or you risk pulling SDK-56 builds).

```
cd apps/mobile
bunx expo install expo-auth-session expo-crypto expo-web-browser expo-secure-store
bun add @supabase/supabase-js
```

Package	Slice	Why
expo-auth-session	A	SMART authorization-code + PKCE flow (ADR-002)
expo-crypto	A	PKCE <code>code_verifier</code> / <code>code_challenge</code> (S256)
expo-web-browser	A	System browser for the Epic login redirect
expo-secure-store	A	Encrypted on-device token storage
@supabase/supabase-js	C	DB client (anon key)

URL polyfill note: `@supabase/supabase-js` on RN often needs `react-native-url-polyfill`. Add it only if you hit a `URL` / `URLSearchParams` error at runtime, and import it once at app entry.

6. Build order & ownership

Three vertical slices, mapped to the `docs/prototype-phases.md` workstreams. They are designed to **not touch the same files**, so they can proceed in parallel and merge independently.

Slice	Deliverable	Primary owner	Touches
A	Epic OAuth (PKCE) + token storage	Bassel (Malek support)	<code>services/epicAuth</code> , <code>services/secureTokens</code> , <code>features/auth</code> , <code>app/connect-epic</code>
B	FHIR client + mapper + seam	Malek (Bassel support)	<code>services/fhir</code> , <code>domain/gameContext</code> , <code>domain/fhirMapper</code> , <code>domain/dataSource</code>
C	Supabase <code>sessions</code> + RLS + client	Malek (Bassel support)	<code>supabase/migrations</code> , <code>services/supabase</code> , <code>features/session</code>
Docs/QA	README updates, PHI checklist, test plan	Summer	<code>docs/*</code> , PR review
UI glue	<code>connect-epic</code> screen styling, wiring snapshot to seam	Humayra	<code>app/*</code> (additively)

Recommended merge order: C → B → A. - **C first** because it has zero UI dependency and unblocks "session row created" early. - **B second** so the seam and `GameContext` type exist for A to populate. - **A last** because it depends on the seam (where to store the connected state) and is the hardest to test (needs the redirect URI registered with Epic).

Each slice has its own deep-dive doc (01–03) and a shared git playbook (04).

7. Resolve these open questions first

From `docs/open-questions.md` — these block specific slices. Raise them at the Thursday client/GTA meeting in Week 1; don't let them stall the seam work (the mock path keeps the demo alive).

#	Question	Blocks	Safe default until answered
1	<code>MedicationRequest</code> vs <code>MedicationStatement</code> ?	Slice B fetch URL + mapper	Fetch both; map whichever returns entries
2	Exact Epic sandbox scopes?	Slice A authorize request	<code>openid fhirUser launch/patient patient/*.read offline_access</code>
3	Fixed sandbox test patient?	Demo script repeatability	Use Epic's published sandbox patient (e.g., Camila Lopez)
7	Epic app public (PKCE) or confidential ?	Slice A token exchange	Assume public ; if confidential, add <code>supabase/functions/epic-token</code>

If #7 comes back "confidential," the token exchange moves server-side into an Edge Function and the client never holds a secret — see Doc 01 §6.

8. What Prototype 1 deliberately does NOT do

Keep scope tight (Definition of Done is narrow on purpose):

- No OpenRouter / quiz generation (that's Prototype 2).
- No `quizzes` / `quiz_responses` / remediation / summary tables yet.
- No Supabase Auth accounts (ADR-004 — lightweight session id only).
- No `packages/shared` yet (introduced in P2 when Edge Functions need shared Zod types).
- No removing the mock — it stays as the default path and the offline demo.

9. Lighter outline of later phases (context only)

So the P1 seams are built with the future in mind:

- **Prototype 2:** `generate-quiz` Edge Function (OpenRouter, server-side); `dataSource`'s `GameContext` is sent to the function; new `quizzes` / `quiz_responses` tables; `packages/shared` for Zod schemas. The mapper from Slice B feeds the function input directly.
- **Final:** `generate-remediation` + `generate-summary` Edge Functions; `remediation_events` and `physician_summaries` tables; summary reads gameplay rows only (ADR-006). The Symptom Check split-card and Session Summary screens already model this behavior in the mock.

10. How to read the rest of this set

Doc	Contents
01 — Epic OAuth (PKCE)	Slice A: design options, file-by-file code sketch, commits, push points
02 — FHIR Client & Mapper	Slice B: the seam, <code>GameContext</code> , mapper, fixtures/tests, commits
03 — Supabase Sessions	Slice C: migration, RLS options, client, session service, commits
04 — Git Workflow & Commit Playbook	Branch model, per-slice commit sequences, when to push, PR gates, 4-week timeline